

OnLive 音: A Browser-Based Real-Time Audio Effects System Using Csound

Lihong Tao¹ and Richard Boulanger

Berklee College of Music

dtao5297@gmail.com; rboulanger@berklee.edu

Abstract. Many digital sound tools are powerful, but the technical barrier to entry in DAWs can make them hard to approach. This paper presents *OnLive 音*, a browser-controlled real-time audio effects console for live input and uploaded samples, built on Csound with a React performance interface. It treats real-time audio processing like a pedalboard rather than a production suite: the user picks an audio source, toggles effects, adjusts controls, and hears the result immediately, without having to build a full digital audio workstation (DAW) session. And every effect can be opened to reveal the Csound .csd file behind it, so users can see exactly how each sound is made and start learning Csound itself. This paper explores how a browser interface can make the depth of Csound directly playable in live performance, through the design and implementation of *OnLive 音*.

Keywords: Real-time audio processing, Csound, Browser-based performance interface, Open Sound Control (OSC), Live performance, Digital signal processing

1 Context and Motivation

OnLive 音 emerged from the first author’s experience as a cellist and electronic digital instrument (EDI) performer. In live sets, she designed and used custom audio effects to extend the cello beyond its familiar acoustic character, producing playful and unexpected timbres. Musicians who heard these performances expressed interest in exploring similar possibilities but often found conventional DAW workflows, including track routing, plugin loading, and audio device configuration, too complex for immediate experimentation. What stopped them was the setup, not the curiosity.

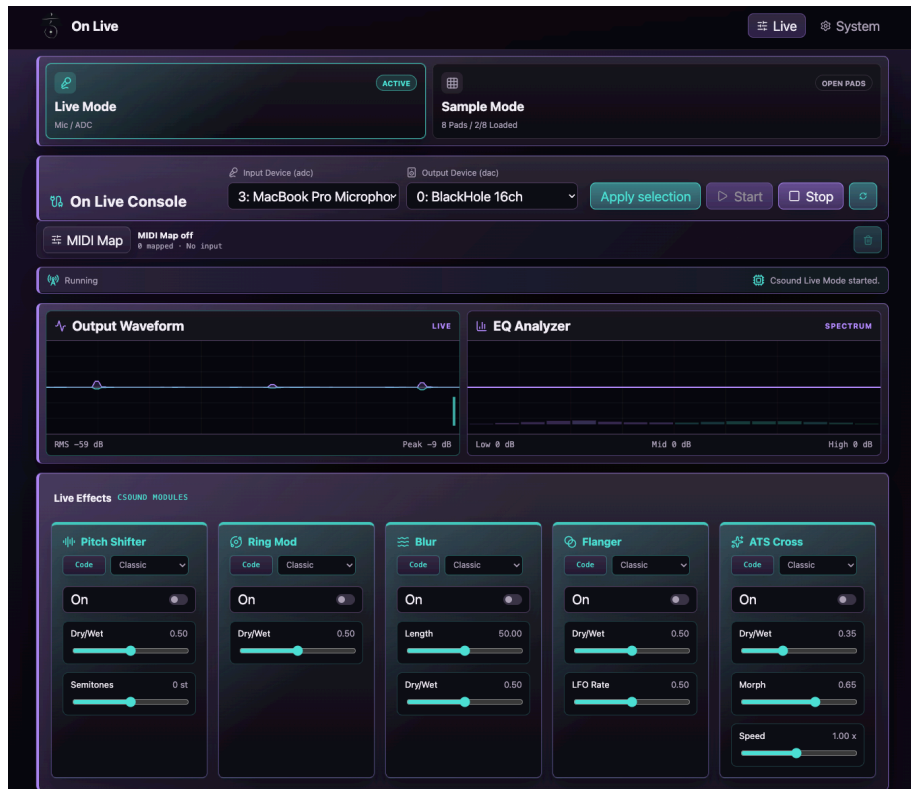


Fig. 1. The *OnLive 音* console in the browser.

¹ Lihong Tao is professionally known as Daphne Tao. Her present affiliation is the Frank J. Guarini School of Graduate and Advanced Studies, Dartmouth College. Source code and supplementary materials for *OnLive 音* are publicly available at <https://github.com/DaphnetaoIII/OnLive>.

OnLive 音 grew out of that gap. It is a browser-controlled audio effects console that lets a performer select a source, start the engine, enable effects, and hear the change immediately, without first setting up a DAW session. The goal is not to replace a DAW, but to serve as an open entry point to real-time sound manipulation, where a musician on any instrument can try live effects for the first time.

A second motivation is educational. Csound is a deep and powerful audio language [1], and like any rich tool, it rewards the time spent learning it. For a beginner, though, a first `.csd` file can be a lot to take in at once, and getting everything configured can delay the moment when you actually hear Csound at work. *OnLive 音* helps with that first step. Each effect can be opened to show the Csound code behind it, and both the code and the surrounding page are kept clean and clearly laid out, so a beginner can see what each part is and what it does. Seeing the code next to the sound it produces makes Csound much easier to start learning. The name reflects this: the character 音 (*yīn*), meaning “*sound*” in Chinese, was borrowed into Japanese as *on*, retaining its original sense. *OnLive 音*, the idea is stated plainly. Put the sound *on*, make it *live*, and let the performer shape it.

2 System Architecture

OnLive 音 is organized into four layers. The React frontend presents the performance controls and receives status updates. The FastAPI backend owns the Csound process, checks control values against predefined ranges, manages sample files, and exposes REST and WebSocket endpoints. OSC carries parameter changes from the backend to Csound on the local machine [5]. Csound itself handles audio input, sample playback, digital signal processing, and output monitoring.

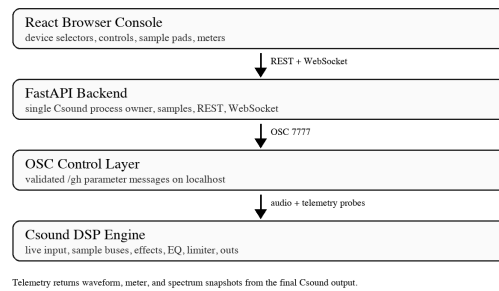


Fig. 2. System architecture.

This separation matters: Csound owns the entire audio path, so the browser serves only as a control and feedback surface, keeping the routing reliable while still allowing a modern, responsive interface.

3 Source Modes and User Workflow

3.1 Live Mode

Live Mode handles direct microphone or instrument input. The interface asks Csound for the available audio devices, parses the ADC and DAC entries, and presents them to the user. After that, the user applies the selection and starts the engine. From that point, changes to effect toggles and sliders are sent to Csound as OSC parameters. Internally, Csound runs at 44.1 kHz with a `ksmps` value of 64 and stereo output. If the input device has only one channel, the signal is copied to stereo. Before any creative effect is applied, the input passes through DC blocking and a high-pass filter. This safety stage reduces low-frequency rumble and helps avoid sudden feedback when a laptop microphone is monitored through speakers.

3.2 Sample Mode

Sample Mode is a second source workflow built around uploaded audio. Users can experiment with samples to find musical ideas, build a performance where pads are triggered at chosen moments to play alongside their own instrument, or test effects without opening a microphone. When Sample Mode is active, the backend starts Csound with zero input channels and only an output device, which removes the risk of feedback entirely.

The sample system uses eight fixed pad slots. Uploaded WAV and MP3 files are converted to 44.1 kHz stereo WAV files for Csound playback. Metadata stores each pad's name, duration, waveform summary, peak level, clipping state, gain, playback mode, loop length, tempo estimate, and effect sends. One-shot

playback is useful for short gestures, while loop playback can stay at the original speed or follow a global BPM. When tempo sync is active, Csound computes the target loop duration from the BPM and bar length; in warp mode, the loop length changes while the pitch stays stable. This allows a loop to match a new tempo without being detuned.

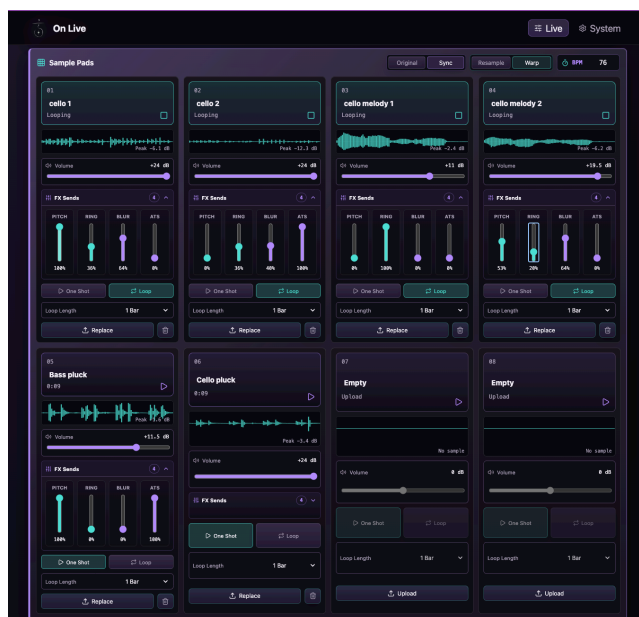


Fig. 3. Sample Mode layout.

4 Csound Audio Engine

4.1 OSC Control and Process Ownership

The backend contains a single CsoundEngine object that owns the live Csound subprocess. This class stores the device selection, current parameter values, sample-mode state, active loop slots, recent logs, and the OSC client. Every public control value is checked against predefined bounds before it is sent to Csound. For example, the pitch control is limited to ± 12 semitones, the EQ bands to ± 12 dB, and the limiter controls are kept within safe limits. When an effect is toggled, its parameters are sent as a single grouped update, so the current slider state is reflected in Csound immediately.

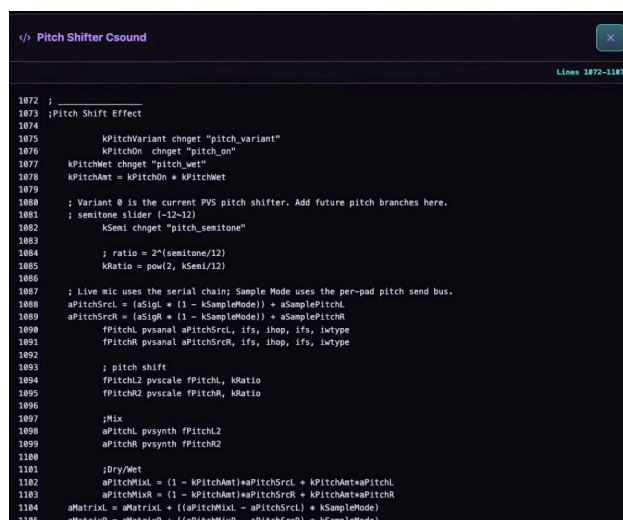


Fig. 4. The effect code dialog, showing the Csound code behind a selected effect.

Inside Csound, an OSC listener handles all messages addressed under `/o1`. The k-rate channels hold the source mode, transport values, sample loop gates, per-pad gains, per-pad FX sends, effect toggles, wet values, EQ gains, master volume, and limiter parameters. This channel design keeps the audio engine independent of the browser: the React interface may change, but Csound only needs a consistent set of channel names and OSC paths.

4.2 DSP Chain

In the current version, the signal runs through five creative effects before reaching the master stage. The **Pitch Shifter** uses a phase vocoder (`pvsanal`, `pvscale`, `pvsynth`) [2, 3], mapping the semitone control to a frequency ratio of $2^{(\text{semitones}/12)}$. **Ring Modulation** multiplies the signal by sine oscillators, slightly detuned between channels for stereo width. **Blur** combines a feedback delay with phase-vocoder analysis and resynthesis [2, 3], producing a soft, smeared texture. **Flanger** uses a short modulated delay, exposing LFO rate and wet amount. **ATS Cross** layers pre-analyzed ATS files [4] as spectral material; because the `ATScross` opcode [2] is file-based, the live or sample signal gates and blends that material rather than feeding it directly.

Live Mode runs these effects as a serial chain; Sample Mode gives each pad a dry bus and per-effect sends, so every pad sets its own send levels before returning to the shared matrix. After the creative effects, the master stage applies **Low**, **Mid**, and **High EQ**: the low band covers energy below 200 Hz, the high band above 5 kHz, and the mid band is formed by subtracting the low and high bands from the full signal. The final bus then applies **DC Blocking**, a low-end guard, soft saturation, master volume, and a look-ahead limiter.

4.3 Output Metering and Visualization

The visual panels are driven by direct measurements of Csound's output rather than by a simulated browser animation. Csound measures the final limited stereo output and sends small OSC metering messages to the backend. Waveform minimum and maximum values are sent at 60 Hz so that peaks are not lost to single-point sampling, while meter and spectrum data are sent at 30 Hz. The spectrum is a compact sixteen-band approximation built from a log-spaced filter bank, light enough for a live, local system. If Csound stops or the data becomes stale, the browser displays an idle baseline instead of simulated motion. This matters for trust: a beginner can see when the engine is silent, when the output is active, and when clipping risk appears.

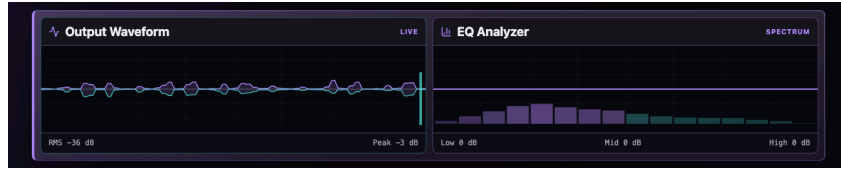


Fig. 5. Output waveform and EQ analyzer. These displays follow Csound's final processed signal.

5 Interface Design, Initial Use, and Limitations

OnLive 音 keeps each control close to a familiar performance action: the mode selector exposes audio routing, the sample pads resemble a performance controller, and the code buttons connect musical controls to their Csound implementation. MIDI mapping further supports hardware control. The first author has used the system in practice and shared it informally with colleagues at Berklee College of Music and Dartmouth College, providing initial observations rather than a structured performer study. Local technical checks confirmed a successful Vite build, Python compilation, Csound syntax, and API health. End-to-end latency has not yet been measured. Formal performer evaluation and round-trip latency testing remain future work.

6 Conclusion and Future Work

OnLive 音 began with a simple wish: that friends who loved live effects could access those sounds without first getting overwhelmed by the routing and configuration of a DAW. The resulting implementation is a browser-based instrument that places the depth of Csound directly under a performer's hands. Its contribution is the path it opens from hearing an effect, to manipulating it in real time, to reading the Csound code behind it. The current version supports live input, sample playback, live effects, MIDI mapping, output metering, and access to the associated Csound code. The system remains under active development. Future work will expand the effect library, support user-supplied `.csd` effect files, measure end-to-end audio latency, and conduct structured performer testing. Plans also include a mobile interface and a hosted version requiring less local setup. The aim is to make live sound easier to enter and to let curiosity determine how deeply a performer explores Csound.

References

1. Boulanger, R.: *The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming*. MIT Press, Cambridge (2000)
2. Csound: *The Canonical Csound Reference Manual*. Csound project documentation, <https://csound.com/docs/manual/>
3. Dolson, M.: The Phase Vocoder: A Tutorial. *Computer Music Journal* **10**(4), 14–27 (1986)
4. Pampin, J.: ATS: A System for Sound Analysis, Transformation and Synthesis Based on a Sinusoidal plus Critical-Band Noise Model and Psychoacoustics. In: *Proceedings of the International Computer Music Conference (ICMC)*. Miami (2004)
5. Wright, M., Freed, A.: Open Sound Control: A New Protocol for Communicating with Sound Synthesizers. In: *Proceedings of the International Computer Music Conference (ICMC)*, pp. 101–104. Thessaloniki (1997)